# Terraform

## OVERVIEW & FUNDAMENTALS



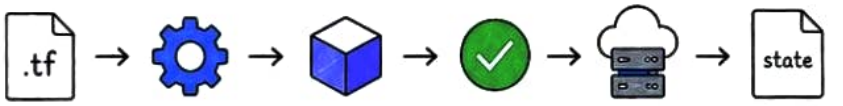
### What is Terraform?

Terraform is an Infrastructure as Code (IaC) tool by HashiCorp that allows you to define, provision and manage infrastructure using human-readable configuration files.

**It helps you:**

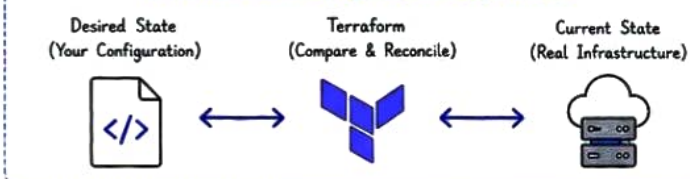
- ✓ Provision infrastructure consistently
- ✓ Multi-cloud and multi-provider support
- ✓ Version infrastructure
- ✓ Automate changes and deployments
- ✓ Manage complex dependencies

### How Terraform Works




1. Write Code (.tf files)
2. terraform init (Initialize)
3. terraform plan (Compare)
4. terraform apply (Apply Changes)
5. Infrastructure Provisioned
6. State File Updated

#### Terraform follows Desired State Model




### Terraform Workflow

**1 Write Configuration**
Define your infrastructure in .tf files using HCL (HashiCorp Configuration Language).

**2 Initialize**
Run 'terraform init' to download providers and initialize the working directory.

**3 Validate**
Run 'terraform validate' to check the syntax and configuration errors.

**4 Plan**
Run 'terraform plan' to preview the changes Terraform will make to reach the desired state.

**5 Apply**
Run 'terraform apply' to create or update infrastructure as per the plan.

**6 Destroy**
Run 'terraform destroy' to destroy the infrastructure managed by Terraform.

### Frequently Used Commands

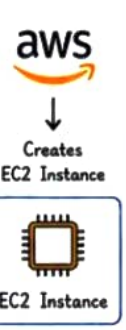
| Command | Purpose | Command | Purpose |
|---|---|---|---|
| terraform init | Initialize a working directory | terraform output | Show output values |
| terraform validate | Validate the configuration files | terraform state list | List resources in the state |
| terraform fmt | Format the configuration files | terraform state rm | Remove a resource from state |
| terraform plan | Show the execution plan | terraform import | Import existing infrastructure |
| terraform apply | Apply the changes | terraform workspace list | List all workspaces |
| terraform destroy | Destroy the infrastructure | terraform workspace new <name> | Create a new workspace |
| terraform show | Show the current state or a plan | terraform workspace select <name> | Switch to a workspace |

### Example Configuration (AWS EC2)

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "web" {
  ami           = "ami-0abcdef1234567890"
  instance_type = "t2.micro"
  tags = {
    Name = "Terraform-EC2"
  }
}

output "public_ip" {
  value = aws_instance.web.public_ip
}
```




### Uses of Terraform

- **Infrastructure Provisioning** — Automate the creation of infrastructure.
- **Multi-Cloud Management** — Manage resources across multiple cloud providers.
- **Consistency & Reusability** — Use code to maintain consistent environments.
- **Collaboration** — Enable teams to work together using version control.
- **CI/CD Integration** — Integrate with CI/CD pipelines for automation.
- **Disaster Recovery** — Recreate infrastructure quickly when needed.

- ✓ Terraform is an IaC tool to manage infrastructure.
- ✓ It uses configuration files to define the desired state.
- ✓ It compares the desired state with the current state and makes necessary changes.
- ✓ It supports multi-cloud, automation, collaboration and versioning.

# Terraform

## STATE MANAGEMENT & DIFFERENCES



### Terraform State File

The state file keeps track of the infrastructure managed by Terraform.

.tfstate

**Stores:**

- Resource IDs
- Resource Attributes
- Dependencies
- Metadata
- Current Infrastructure State

**Terraform compares**

| Configuration (Desired State) | ⟷ | State File (Current State) | ⟷ | Real Infrastructure (Actual State) |
|---|---|---|---|---|

### Local vs Remote State

**Local State**

terraform.tfstate

Stored on your local machine.

**Cons:**

- ✗ Not suitable for teams
- ✗ No state locking
- ✗ Risk of state loss
- ✗ No backup/versioning

**Remote State**

Stored in a remote backend (S3, Azure Blob, GCS or Terraform Cloud).

**Benefits:**

- ✓ Team Collaboration
- ✓ State Locking
- ✓ Versioning & Backup
- ✓ High Availability
- ✓ Better Security

### Remote State Backends (Examples)

| AWS S3 (with DynamoDB for locking) | Microsoft Azure Storage Account (with Blob) | Google Cloud Storage | Terraform Cloud (Managed Backend) | Other Options Consul, etcd, etc. |
|---|---|---|---|---|

### State File Corruption – Causes

- Manual editing of state file
- Interrupted "terraform apply"
- Simultaneous execution without locking
- Accidental deletion of state file
- Backend/storage issues

### State File Corruption – Recovery Steps

1. Restore from backend versioning/backup (if available).
2. If infrastructure still exists, re-import using "terraform import".
3. Rebuild state using "terraform state" commands if needed.
4. Run "terraform plan" to verify the recovered state.

### Workspaces

Workspaces allow you to manage multiple isolated state files within the same configuration.

default (Default workspace)
dev
qa
uat
prod

Each workspace has its own state file.

**Useful For:**

- ✓ Environments (Dev, QA, Prod)
- ✓ Separate Teams
- ✓ Isolated Deployments

**Common Commands**

```
> terraform workspace list              # List all workspaces
> terraform workspace new <name>        # Create new workspace
> terraform workspace select <name>     # Switch workspace
> terraform workspace delete <name>     # Delete workspace
```

### Terraform State Commands

| Command | Purpose |
|---|---|
| terraform state list | List all resources in the state |
| terraform state show <addr> | Show details of a resource |
| terraform state rm <addr> | Remove resource from state |
| terraform import <addr> <id> | Import existing resource |
| terraform state mv <a> <b> | Move resource in state |
| terraform refresh | Refresh state with real infra (legacy, use plan/apply now) |

**Key Takeaways**

- State file is the heart of Terraform.
- Always use a remote backend for team and production environments.
- Enable state locking and versioning to prevent corruption and data loss.
- Workspaces help you maintain separate environments with isolated states.
- Regular backups of the state file are critical for disaster recovery.

**Best Practice**

Never edit state file manually. Always use Terraform commands!

# Terraform

## INTERVIEW QUESTIONS & REAL-TIME SCENARIOS



### 1 What happens if the Terraform state file is deleted?

- Terraform loses track of the infrastructure.
- Resources may still exist, but Terraform no longer manages them.

**Recovery:**

- Restore state from backend/backup.
- If no backup, re-import existing resources using "terraform import".

### 2 Using workspaces, if the state file is deleted, how do you recover?

default
dev
qa
uat
prod

- Select the correct workspace.
- If using remote backend, restore the state from versioning/backup.
- If no backup, re-import existing resources.

```
terraform workspase select prod
terraform plan
terraform import <resource> <id>
terraform apply
```

### 3 What if the state file is corrupted?

- Restore from backup or backend versioning.
- Validate the state.
- Run "terraform plan" to check for drift.
- Re-import resources if required.

### 4 How do you prevent state corruption?

- Use a remote backend.
- Enable state locking (e.g., DynamoDB with S3).
- Enable versioning on backend storage.
- Never edit state file manually.
- Restrict write access to the state.

### 5 Difference between Local State and Remote State?

| | Local State | Remote State |
|---|---|---|
| Storage | Stored on local machine | Stored in remote backend |
| Team Usage | Not suitable for teams | Suitable for teams |
| State Locking | No locking | Supports locking |
| Versioning | No versioning | Versioning available |
| Risk | High risk of loss | Low risk (backup & versioning available) |

### 6 What is "terraform import"?

Imports existing infrastructure into Terraform state without recreating resources.

```
terraform import aws_instance.web
 i-1234567890abcdef0
```

### 7 What is State Locking?

State locking prevents multiple users from modifying the state simultaneously, avoiding conflicts and corruption.

### 8 What are Terraform Modules?

Modules are reusable collections of Terraform configurations. They help in code reusability, standardization and easier management.

**Example:**

```
modules/
  └── network/
  ├── compute/
  └── database/
```

### 9 Difference between "terraform plan" and "terraform apply"?

- "terraform plan": Shows the changes Terraform will make (preview).
- "terraform apply": Executes the changes and updates the infrastructure.

### 10 Real-Time Scenario: Your laptop crashed while running "terraform apply". What would you do?


1. Check if the state was updated.
2. Verify the actual cloud resources.
3. Restore the state from backend/backup if needed.
4. Run "terraform plan" to identify any differences.
5. If resources exist but missing from state, use "terraform import".
6. Run "terraform apply" again after confirming the plan.

**Commands Flow**

terraform plan
↓
Review Changes
↓
terraform apply

### Best Practices

- ✓ Always use a remote backend.
- ✓ Enable state locking and versioning.
- ✓ Never modify state file manually.
- ✓ Use workspaces for environment isolation.
- ✓ Regularly backup your state files.
- ✓ Review "terraform plan" before every apply.

### Key Takeaways

- ✓ State file is the single source of truth for Terraform.
- ✓ Remote state ensures collaboration, safety and reliability.
- ✓ Workspaces help manage multiple isolated environments.
- ✓ Use import and state commands to recover infrastructure.
- ✓ Always follow best practices for secure and stable IaC.

# Azure API Management (APIM)
## FUNDAMENTALS

PAGE 1

## What is APIM?

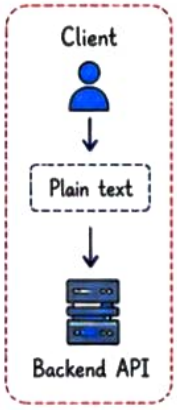
Azure API Management (APIM) is a fully managed Azure service that acts as a centralized gateway for APIs.

It helps organizations:

- ✓ Publish APIs
- ✓ Secure APIs
- ✓ Monitor API usage
- ✓ Transform requests/responses
- ✓ Manage API lifecycle
- ✓ Expose APIs to internal and external consumers

## Why APIM?

### Without APIM:




**Problems:**

- ✗ No central security
- ✗ No throttling
- ✗ No monitoring
- ✗ Hard to manage multiple APIs

### With APIM:


**Benefits:**

- ✓ Security
- ✓ Scalability
- ✓ Governance
- ✓ Monitoring
- ✓ API Reuse

## APIM Core Components

### Gateway

Main entry point for API requests.

**Responsibilities:**

- Authentication
- Authorization
- Routing
- Request validation
- Rate limiting

### APIs

Collection of operations exposed through APIM.

**Example:**

- Product API
- Order API
- Payment API
- Customer API

### Products

Logical grouping of APIs.

**Example:**

- Gold Product
  - Product API
  - Order API
  - Payment API
- Silver Product
  - Product API
  - Order API

### Developer Portal

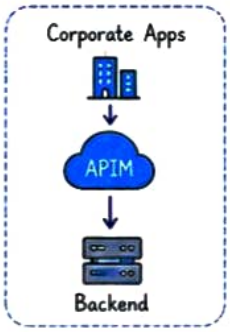
Self-service portal for developers.

**Features:**

- API documentation
- API testing
- Subscription management
- SDK downloads

## APIM Deployment Modes

### Internal Mode




Only accessible inside VNET.

**Used for:**

- Enterprise APIs
- Internal Applications
- Microservices

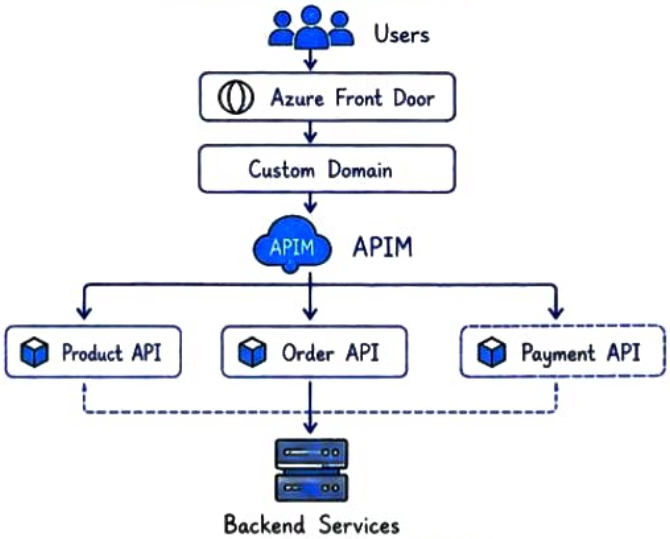
### External Mode


Publicly accessible.

**Used for:**

- Partner APIs
- Customer APIs
- Mobile Apps

## APIM Architecture




- ✓ APIM is the front door for APIs
- ✓ Centralized API governance
- ✓ Supports internal and external APIs
- ✓ Simplifies security and monitoring
- ✓ Reduces backend exposure

APIM helps you build secure, scalable and well-governed APIs!

# APIM Security, Networking & Integration

PAGE 2

## APIM Security

### Authentication Methods

#### OAuth 2.0

Industry standard authentication.


#### JWT Validation

APIM validates JWT tokens before forwarding requests.

Policy: XML

```
<validate-jwt>
  ...
</validate-jwt>
```

Benefits:

- ✓ Secure APIs
- ✓ Claims validation
- ✓ Role-based access

### Subscription Keys

APIM generated key:

Ocp-Apim-Subscription-Key

Example:

```
HTTP GET /products
Ocp-Apim-Subscription-Key: xxxxx
```

### Named Values

Stores configuration securely.

Examples:

- Client ID
- Client Secret
- Backend URL
- API Keys
- Environment Variables

Instead of: `https://prod.company.com`

Use: `{{backend-url}}`

### Managed Identity

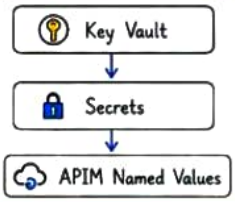
APIM can access Azure services without storing secrets.

Example:


Benefits:

- ✓ No password storage
- ✓ Secure authentication
- ✓ Azure-native security

### APIM + Azure Key Vault



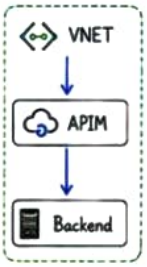

Store:

- Client Secret
- Certificates
- Connection Strings
- API Keys

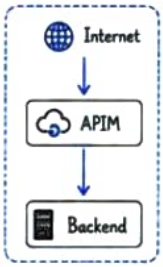
## Networking

### Internal vs External

Internal Private APIs

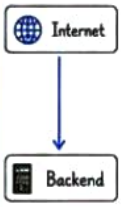



External Public APIs

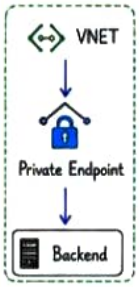



### Private Endpoints

Without Private Endpoint:




With Private Endpoint:

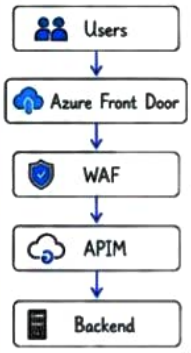



Benefits:

- ✓ No public exposure
- ✓ Secure communication
- ✓ Compliance requirements

### APIM with Azure Front Door

Architecture:




Front Door Responsibilities:

- ✓ Global routing
- ✓ SSL termination
- ✓ WAF
- ✓ Load balancing
- ✓ DDoS protection

### NAT Gateway

Used for:

- Static outbound IP
- Secure outbound traffic


### APIM + Azure Functions


Benefits:

- ✓ Hide Function URL
- ✓ Secure Access
- ✓ Monitoring
- ✓ Policies

## Security Best Practices

- ✓ Use OAuth 2.0
- ✓ Validate JWT
- ✓ Enable Private Endpoints
- ✓ Store secrets in Key Vault
- ✓ Use Managed Identity
- ✓ Use Front Door + WAF
- Enable IP Filtering

# Azure API Management (APIM)

PAGE 3 – POLICIES, REQUEST FLOW, MONITORING & INTERVIEW QUESTIONS

APIM

## APIM Policy Pipeline

Client Request
↓
Inbound Policies
↓
Backend Policies
↓
Backend Service
↓
Outbound Policies
↓
On Error Policies
↓
Response

### Inbound Policies

Executed before backend call.

Examples:

| | |
|---|---|
| Validate JWT | <validate-jwt> |
| Rate Limiting | <rate-limit-by-key> |
| Rewrite URL | <rewrite-uri> |
| Check Headers | <check-header> |

### Backend Policies

Executed before reaching backend.

Examples:

- ✓ Load balancing
- ✓ Backend switching
- ✓ Retry logic
- ✓ Circuit breaker

### Outbound Policies

Executed after backend response.

Examples:

- ✓ Response transformation
- ✓ Header manipulation
- ✓ Data masking
- ✓ XML to JSON

### Error Policies

Examples:

- ✓ Custom error messages
- ✓ Logging
- ✓ Retry
- ✓ Fallback response

## Request Lifecycle

Client
↓
APIM Gateway
↓
Authentication
↓
Authorization
↓
Policies
↓
Backend Service
↓
Response Transformation
↓
Client

## Monitoring & Analytics

### Azure Monitor

Provides:

- ✓ Metrics
- ✓ Alerts
- ✓ Dashboards
- ✓ Health Monitoring

### Application Insights

Tracks:

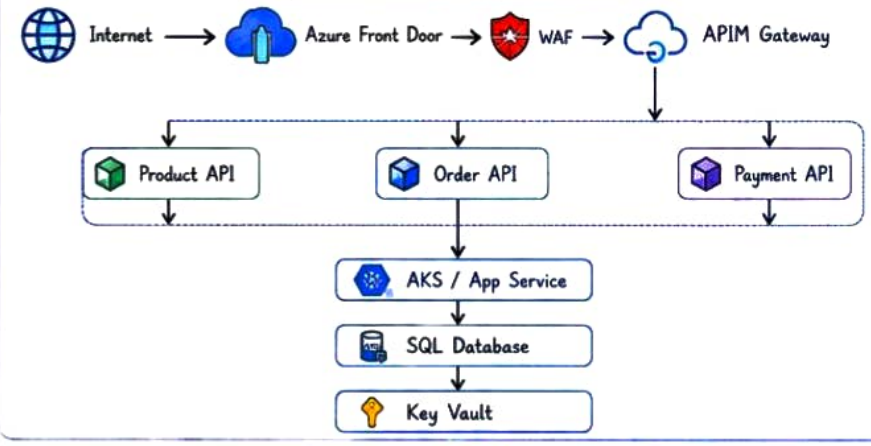
- ✓ Requests
- ✓ Dependencies
- ✓ Exceptions
- ✓ Performance
- ✓ Availability

### APIM Analytics

Shows:

- ✓ Top APIs
- ✓ Response Times
- ✓ Failure Rates
- ✓ Consumer Usage
- ✓ Subscription Metrics

## Real Enterprise Architecture




## Common HTTP Status Codes

| Code | Meaning |
|---|---|
| 200 | Success |
| 201 | Created |
| 204 | No Content |
| 400 | Bad Request |
| 401 | Unauthorized |
| 403 | Forbidden |
| 404 | Not Found |
| 409 | Conflict |
| 429 | Too Many Requests |
| 500 | Internal Server Error |
| 502 | Bad Gateway |
| 503 | Service Unavailable |
| 504 | Gateway Timeout |

## APIM Interview Questions

**Q1. What is APIM?**
A centralized Azure service used to publish, secure, monitor and manage APIs.

**Q2. What are Products?**
Logical grouping of APIs exposed to consumers.

**Q3. What are Policies?**
Rules applied to requests and responses.
Examples:

- Authentication
- Rate Limiting
- Transformation
- Routing

**Q4. What is Named Value?**
Reusable configuration value stored in APIM.

**Q5. Difference between Internal and External APIM?**
Internal = Private access only.
External = Internet accessible.

**Q6. Why use Front Door with APIM?**
Provides WAF, global routing, SSL termination and DDoS protection before APIM.

**Q7. What is Subscription Key?**
Unique key used by consumers to access APIs.

**Q8. What is the APIM Request Pipeline?**
Inbound → Backend → Outbound → Error Handling.

## Final APIM Formula

APIM = Gateway + Security + Policies + Monitoring + Developer Portal + Analytics + API Governance

★ FOLLOW ME FOR MORE SUCH NOTES ★

# TERRAFORM

## SCENARIO BASED INTERVIEW QUESTIONS & ANSWERS

★ From Basic to Advanced ★

### 1 What is Terraform?

**Scenario:** Explain Terraform to a beginner.

**Answer / Solution:**

- Terraform is an Infrastructure as Code (IaC) tool by HashiCorp.
- It helps to provision, manage and version infrastructure.
- It supports multiple cloud providers.

### 2 Explain terraform workflow.

**Scenario:** How will you create infrastructure using Terraform?

**Answer / Solution:**

Write Code (Configuration) → terraform init (Initialize) → terraform plan (Preview)
↳ terraform apply (Create Infra) → terraform destroy (Destroy Infra)

### 3 What is the difference between terraform plan and terraform apply?

**Scenario:** You want to preview changes before applying.

**Answer / Solution:**

- terraform plan → Shows the execution plan.
- terraform apply → Applies the changes to create/update resources.

### 4 What is the purpose of terraform.tfstate file?

**Scenario:** Why is state file important?

**Answer / Solution:**

- It tracks the current state of infrastructure.
- Terraform uses it to map real resources.
- Never edit it manually.

### 5 What are Terraform providers?

**Scenario:** How does Terraform interact with cloud platforms?

**Answer / Solution:**

- Providers are plugins that allow Terraform to interact with APIs of cloud platforms.
- Example: aws, azurerm, google, kubernetes.

### 6 How do you handle secrets in Terraform?

**Scenario:** You need to store database password securely.

**Answer / Solution:**

- Use environment variables.
- Use terraform.tfvars (gitignored).
- Use secret management tools (AWS Secrets Manager, Azure Key Vault, HashiCorp Vault).

### 7 How do you manage multiple environments (Dev, QA, Prod)?

**Scenario:** You need separate environments with different configurations.

**Answer / Solution:**

- Use workspaces.
- Or use separate directories with different tfvars files.
- Use variables for environment-specific values.

### 8 How do you handle Terraform state?

**Scenario:** State file is important. How will you manage it?

**Answer / Solution:**

- Store state remotely (S3 + DynamoDB, Azure Storage, GCS).
- Enable state locking to prevent concurrent updates.
- Backup the state file regularly.

### 9 How do you refactor Terraform code?

**Scenario:** Your Terraform code is big and hard to manage.

**Answer / Solution:**

- Use modules to break code into reusable components.
- Use variables and outputs properly.
- Follow DRY principle.

### 10 How do you import existing infrastructure into Terraform?

**Scenario:** You have existing resources and want to manage using Terraform.

**Answer / Solution:**

- Use 'terraform import <resource_type.name> <resource_id>'
- Then run 'terraform plan' and 'terraform apply'.

### IMPORTANT COMMANDS

| | |
|---|---|
| terraform init | – Initialize working directory |
| terraform plan | – Show execution plan |
| terraform apply | – Apply changes |
| terraform destroy | – Destroy infrastructure |
| terraform validate | – Validate configuration |
| terraform fmt | – Format code |
| terraform import | – Import existing resource |
| terraform output | – Show output values |

### BEST PRACTICES

- ✓ Write modular and reusable code
- ✓ Use remote state with locking
- ✓ Keep state file secure
- ✓ Use variable configuration values
- ✓ Follow naming conventions
- ✓ Review plan before apply
- ✓ Automate using CI/CD pipelines

### COMMON TERRAFORM FILES

| | |
|---|---|
| main.tf | – Main configuration |
| variables.tf | – Input variables |
| outputs.tf | – Output values |
| providers.tf | – Provider configuration |
| terraform.tfvars | – Variable values |
| versions.tf | – Terraform & provider versions |

**EXAM TIP:** Always understand the scenario, break it down, and write clean, modular, and maintainable Terraform code.

Handwritten Notes ✱ By- Parisha Singhal ♡

# TERRAFORM with AZURE

## Infrastructure as Code (IaC)

### 1. What is Terraform?

- Terraform is an open-source IaC tool by HashiCorp.
- It allows us to define and provision infrastructure across multiple cloud providers using code.
- For Azure, it uses Azure Provider to manage Azure resources.

Terraform

### 2. Why Terraform?

- ✓ Infrastructure as Code
- ✓ Version Controlled (Git)
- ✓ Repeatable & Consistent
- ✓ Eliminates Manual Errors
- ✓ Reusable Modules
- ✓ Multi-cloud Support
- ✓ Easy Integration with CI/CD

### 3. Azure Resources we can manage using Terraform

- Resource Groups
- Virtual Networks (VNet)
- Subnets
- Network Security Groups (NSG)
- Virtual Machines (Windows / Linux)
- Public IP Addresses
- Load Balancers
- Azure Storage Accounts
- Azure Key Vault
- Azure App Service
- Azure SQL Database
- Azure Kubernetes Service (AKS)
- Virtual Machine Scale Sets (VMSS)

### 4. Core Terraform Concepts

- Providers – Define cloud platform
- Resources – The infrastructure
- Variables – Input values
- Outputs – Output values
- Modules – Reusable code
- State File – terraform.tfstate (Keeps track of resources)
- Backend – Remote storage for state file
- Data Sources – Fetch info from existing resources
- Workspaces – Multiple environments

### 5. Terraform Workflow

Write Infrastructure Code (.tf files)

↓

terraform init → Initialize Terraform & download providers

↓

terraform validate → Validate the configuration

↓

terraform plan → Preview the changes

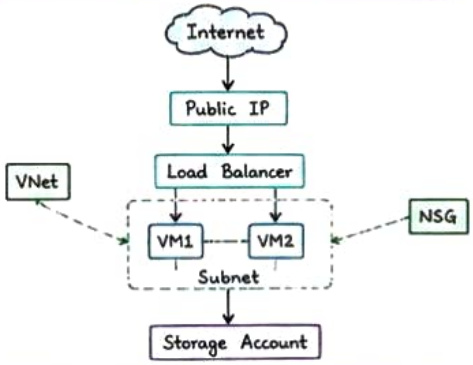
↓

terraform apply → Create / Update Infrastructure

↓

terraform destroy → Destroy Infrastructure (when needed)

### 6. Example: Azure Infrastructure




### 7. Sample Terraform Structure

```
terraform-azure/
├── main.tf            # Main configuration
├── variables.tf       # Input variables
├── outputs.tf         # Output values
├── providers.tf       # Provider configuration
├── terraform.tfvars   # Variable values
└── modules/           # Reusable modules
```

### 8. Benefits in Real World

- ♥ Automates Azure Infrastructure
- ♥ Saves Time & Effort
- ♥ Ensures Consistency
- ♥ Easy Rollback & Versioning
- ♥ Scalable & Production Ready
- ♥ Works Great with Azure DevOps and GitHub Actions

### 9. Common Azure Use Cases with Terraform

- ★ Deploy Networking (VNet, Subnets, NSG)
- ★ Provision Virtual Machines
- ★ Create Storage Accounts
- ★ Deploy AKS Clusters
- ★ Manage Key Vaults
- ★ Configure Load Balancers
- ★ Setup App Service
- ★ Create SQL Databases
- ★ Manage RBAC & IAM
- ★ Build Complete Environments (Dev, Test, Prod)

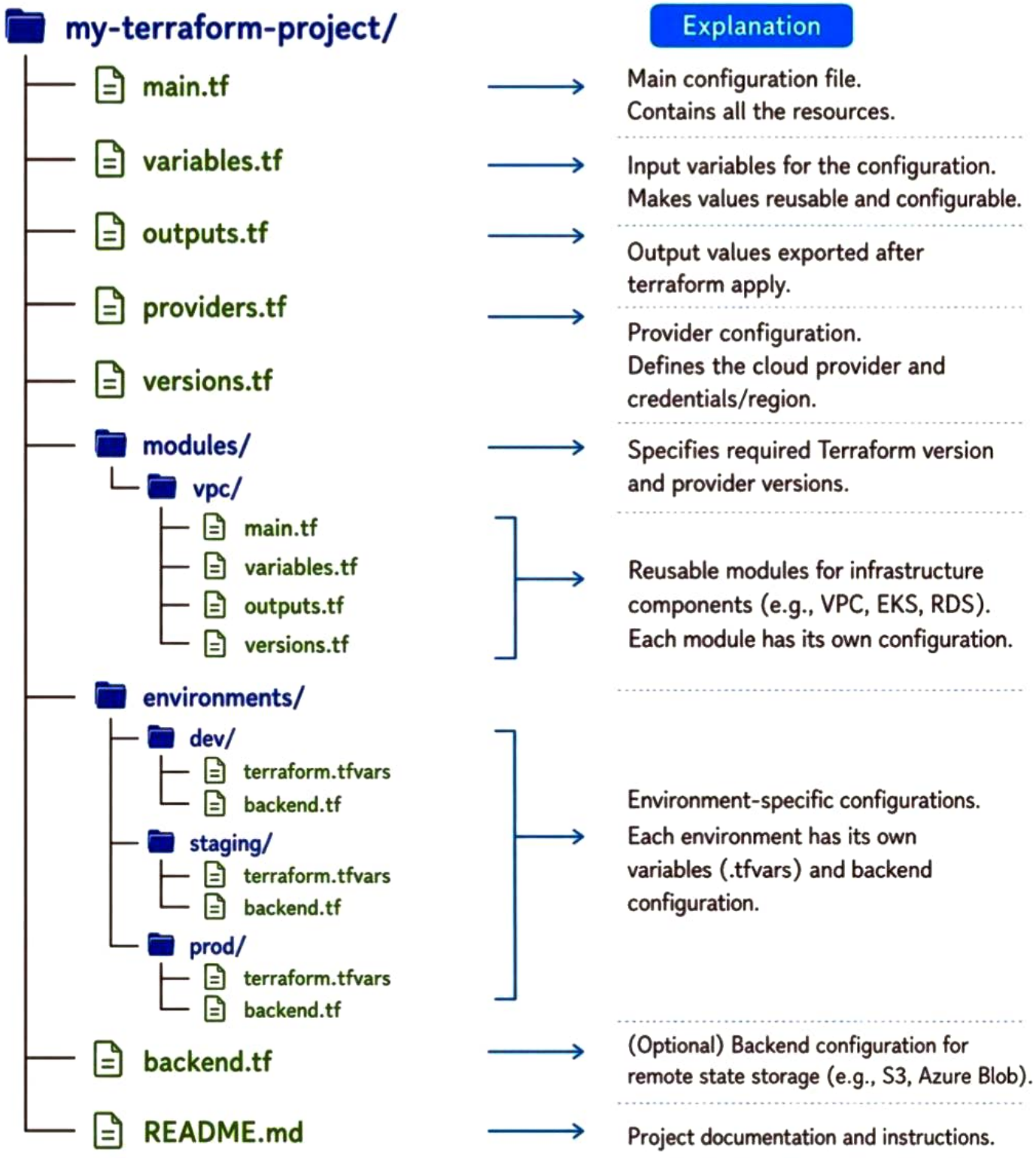
### Note to Self

- Infrastructure as Code is not just about tools, it's about building a better way to manage cloud infrastructure. ♥

Keep Learning. Keep Building. Keep Automating.

By- Parisha Singhal ♡

# Terraform Project Structure



# COMPLETE NETWORKING (as per DevOps Engineer)

## 1. OSI Model (7 Layers)

7. Application – HTTP, FTP, SSH, DNS
6. Presentation – SSL/TLS, Encoding
5. Session – Session Mgmt, API
4. Transport – TCP, UDP
3. Network – IP, ICMP, Routing
2. Data Link – MAC, Switch, ARP
1. Physical – Cables, Hubs, Signals

## 2. IP Addressing

- IPv4 – 32 bit (Example: 192.168.1.1)
- IPv6 – 128 bit
- Public IP – Internet facing
- Private IP – Internal use (Not routable on internet)
  - 10.0.0.0 – 10.255.255.255
  - 172.16.0.0 – 172.31.255.255
  - 192.168.0.0 – 192.168.255.255
- Subnet Mask – Defines network & host
- CIDR Notation – 192.168.1.0/24

## 3. Important Protocols

- TCP (Transmission Control Protocol)
  - Connection Oriented, Reliable, Slow
  - Used by: HTTP, HTTPS, SSH, FTP
- UDP (User Datagram Protocol)
  - Connectionless, Fast, Unreliable
  - Used by: DNS, DHCP, Streaming
- ICMP (Internet Control Message Protocol)
  - Used for error reporting & diagnostics
  - Example: ping, traceroute
- DNS (Domain Name System)
  - Converts domain name to IP address
  - Uses UDP/TCP port 53
- DHCP (Dynamic Host Configuration Protocol)
  - Assigns IP address automatically
  - Uses UDP port 67 (server), 68 (client)
- HTTP/HTTPS – Web traffic (80/443)
- SSH – Secure Remote Access (22)
- FTP – File Transfer (20, 21)
- SMTP/IMAP/POP3 – Email (25, 143, 110)

## 4. Ports

| Port | Protocol | Used For |
|---|---|---|
| 20,21 | TCP | FTP |
| 22 | TCP | SSH |
| 53 | UDP/TCP | DNS |
| 80 | TCP | HTTP |
| 443 | TCP | HTTPS |
| 3306 | TCP | MySQL |
| 5432 | TCP | PostgreSQL |
| 6379 | TCP | Redis |

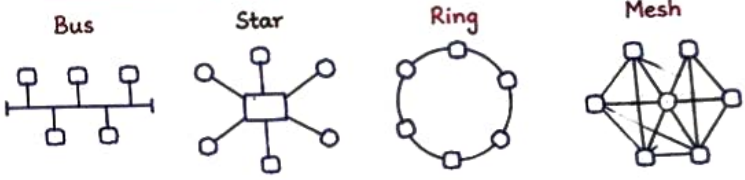
## 5. Networking Devices

- Hub – Broadcasts data to all ports (Layer 1)
- Switch – Sends data to specific port (Layer 2)
- Router – Forwards packets between networks (Layer 3)
- Firewall – Monitors & controls traffic
- Load Balancer – Distributes traffic across servers
- Gateway – Entry/Exit point of a network

## 6. Common Network Commands

- ip a / ifconfig – Check IP & interfaces
- ping <IP/Domain> – Check connectivity
- traceroute <IP> – Trace packet path
- nslookup <domain> – DNS lookup
- netstat -tulnp – Check open ports
- curl <url> – Test HTTP/HTTPS
- dig <domain> – DNS query
- arp -a – ARP table
- route -n – Routing table

## 7. Network Topologies




## 8. Cloud Networking (DevOps Perspective)

- VPC (Virtual Private Cloud) – Isolated cloud network
- Subnet – Divides VPC into smaller networks
  - Public Subnet – Has route to Internet Gateway
  - Private Subnet – No direct internet access
- Internet Gateway – Allows communication between VPC and Internet
- NAT Gateway – Allows outbound internet from private subnet
- Route Table – Controls traffic flow
- Security Group – Acts as virtual firewall (Stateful)
- Network ACL – Firewall at subnet level (Stateless)
- Peering – Connect two VPCs privately
- Load Balancer – Distributes incoming traffic

## 9. Important Concepts

- Latency – Time taken for data to travel
- Bandwidth – Amount of data transfer capacity
- Throughput – Actual data transfer rate
- Packet – Small unit of data in network
- MTU – Maximum Transmission Unit
- QoS – Quality of Service

## 10. Common Use in DevOps

- CI/CD tools communication (Jenkins, GitHub Actions)
- App deployment (Docker, Kubernetes)
- Monitoring & Logging (Prometheus, Grafana, ELK)
- Cloud Resources connectivity (AWS/Azure/GCP)
- Troubleshooting (ping, traceroute, logs)

★ Remember:
Good Networking knowledge is essential for Troubleshooting, Security & Performance in DevOps.

– By Abhishek Singh

# AZURE CLOUD MINDMAP

## A practical guide for Data + Cloud Engineers

Aiswarya Venkitesh

DON'T FORGET TO SAVE

### 1 Compute

Virtual Machines | VM Scale Sets | App Service | Batch | Azure Functions | AKS | Container Instances (ACI) | Service Fabric

### 2 Storage

Blob Storage | ADLS Gen2 | File Storage | Queue Storage | Table Storage | Managed Disks | Hot / Cool / Archive tiers

### 3 Databases

Azure SQL Database | SQL Managed Instance | Cosmos DB | PostgreSQL / MySQL Managed | Synapse SQL Pools | Redis Cache

### 4 Data & Analytics

Azure Data Factory | Synapse Analytics | Databricks | Event Hubs | Stream Analytics | Data Explorer (Kusto) | Purview | Power BI

### 5 Networking

VNet | Subnets | NSG | DNS | Private Endpoints | VPN Gateway | ExpressRoute | Load Balancer | Application Gateway | Azure Firewall | Front Door / Traffic Manager

### 6 Integration & Messaging

Service Bus

### 7 Security & Identity

Entra ID (Azure AD) | RBAC | Managed Identity | Key Vault | Defender for Cloud | Azure Policy | MFA

### 8 Monitoring & Ops

Azure Monitor | Log Analytics | Application Insights | Alerts | Workbooks | Event Grid

### 9 DevOps & IaC

Azure DevOps | GitHub Actions | ARM Templates | Bicep | Terraform | Cost Management | SLA / Reliability basics | Advisor

### 10 Governance & Best Practices

Resource Groups | Management Groups | Naming conventions | Tags | Landing Zones | Backup / DR basics | Compliance basics

## AZURE FRONT DOOR

Modern Layer 7 (Application) Load Balancer at Microsoft Global Edge.

Provides security, performance & intelligent routing.

VS

## AZURE TRAFFIC MANAGER

DNS-based Global Load Balancer.

Routes user traffic to the best or healthy endpoint using DNS responses.

| FEATURE | AZURE FRONT DOOR | AZURE TRAFFIC MANAGER |
|---|---|---|
| Service Type | Layer 7 (Application Layer) | Layer 3/4 (DNS Layer) |
| Routing | HTTP/HTTPS based routing (Path, Host, Header, Query String, etc.) | DNS based routing (Priority, Performance, Weighted, Geographic, Multi-value) |
| Global Presence | Uses Microsoft global edge network (Anycast) 200+ POPs worldwide | Uses DNS resolution Depends on DNS propagation (TTL) |
| Performance | Low latency, Fast, Intelligent routing Traffic reaches Microsoft edge first | Relatively higher latency Depends on DNS resolution time |
| Health Probing | Active health probes at the edge Very fast failure detection (~seconds) | Health probes from Traffic Manager Slower failover (TTL dependent, ~minutes) |
| Protocols Supported | HTTP, HTTPS (Layer 7 features supported) | Any Protocol (TCP/UDP/HTTP/HTTPS) (Only DNS routing) |
| Security | Built-in WAF, DDoS Protection, TLS Termination, Bot Protection, Rate Limiting | No WAF, No TLS Termination Basic DDoS Protection |
| Caching | Yes – Global Caching (Static & Dynamic content) | No |
| Use Cases | Web applications, APIs, SaaS, Microservices, Global secure apps | Non-HTTP services, Legacy apps, Simple DNS based routing |
| Cost | Higher (Premium features) | Lower (Cost effective) |
| Best When | You need security, performance, intelligent routing and modern app delivery | You need simple, cost-effective, DNS based traffic distribution |

## KEY TAKEAWAYS

### Use Azure Front Door

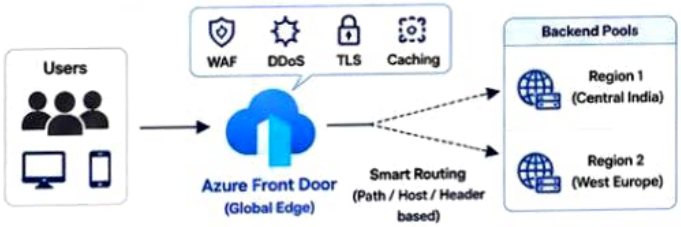
- ✓ For modern web apps & APIs
- ✓ For security (WAF, DDoS, Bot Protection)
- ✓ For performance & low latency
- ✓ For advanced routing requirements

### Use Azure Traffic Manager

- ✓ For non-HTTP services (TCP/UDP/SMTP etc.)
- ✓ For simple DNS based routing
- ✓ For cost-effective global failover scenarios
- ✓ For legacy applications

## ARCHITECTURE COMPARISON

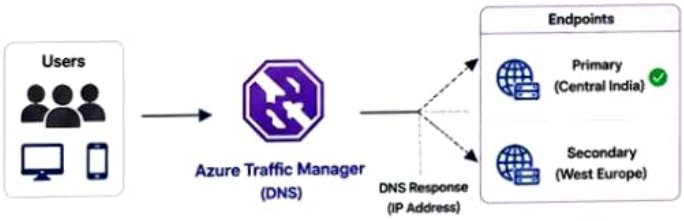
### AZURE FRONT DOOR – LAYER 7 ROUTING (EDGE)




**EXAMPLE:**

If user requests https://app.contoso.com/api
→ Front Door checks rule (Path: /api)
→ Routes to API backend in Central India
If /web → Routes to Web backend in West Europe

### AZURE TRAFFIC MANAGER – DNS ROUTING




**EXAMPLE:**

User requests https://app.contoso.com
→ Traffic Manager DNS returns IP of Healthy endpoint
→ If Central India is down, returns West Europe IP
→ User connects to West Europe

## SUMMARY

### Front Door

- Advanced routing (L7)
- Global edge network
- Security (WAF, DDoS, Bot Protection)
- Low latency & high performance
- Best for modern, secure web applications

Use The Right Service!

### Traffic Manager

- ✓ DNS based routing (L3/4)
- ✓ Simple & cost-effective
- ✓ Supports any protocol
- ✓ Slower failover (TTL based)
- ✓ Best for non-HTTP or legacy applications

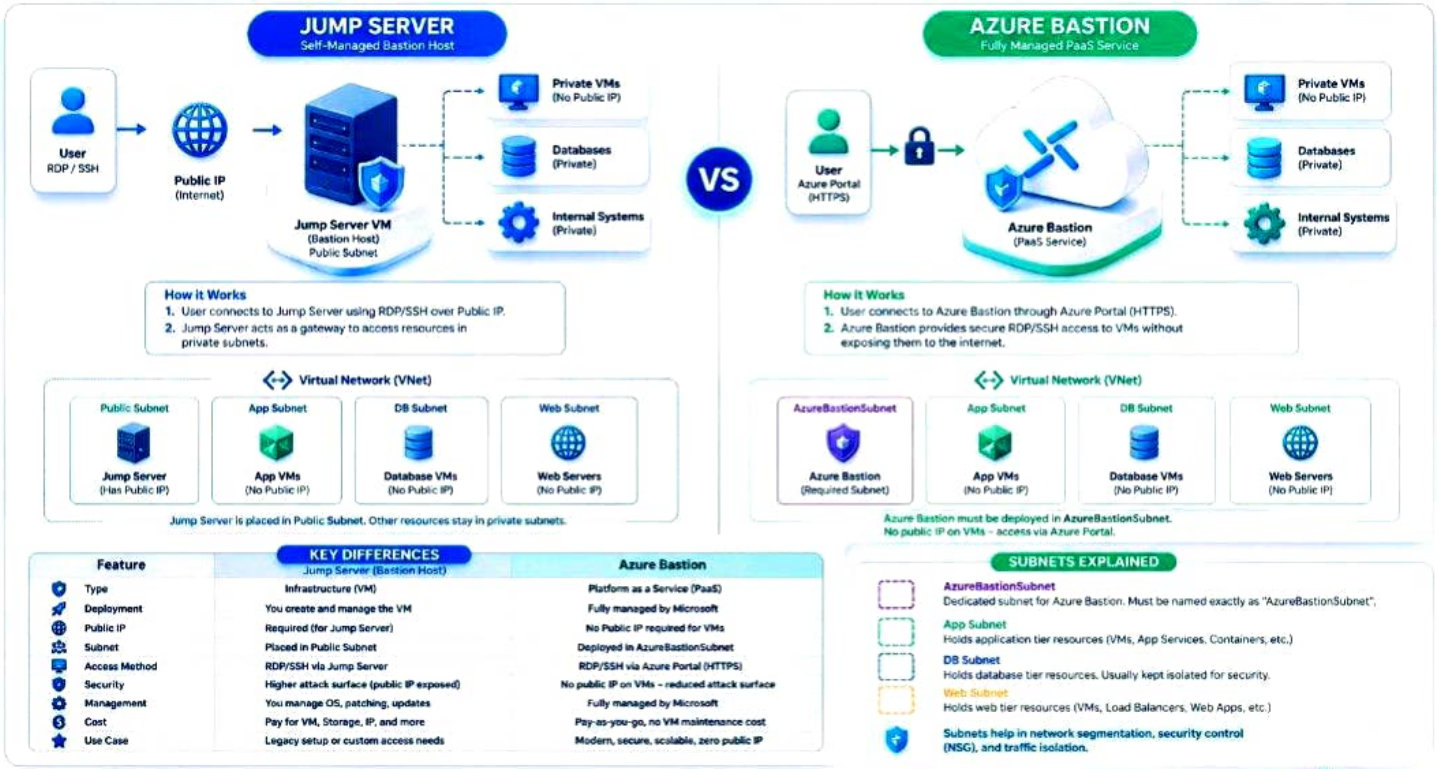
## QUICK EXAMPLE

You have a Banking Web App:
→ Use Front Door → for security, speed and intelligent routing.

You have a Game Server (TCP):
→ Use Traffic Manager → for simple DNS based failover.

# Jump Server VS Azure Bastion

Secure Access to Your Azure Resources – Which One Should You Use?




## KEY DIFFERENCES

| Feature | Jump Server (Bastion Host) | Azure Bastion |
|---|---|---|
| Type | Infrastructure (VM) | Platform as a Service (PaaS) |
| Deployment | You create and manage the VM | Fully managed by Microsoft |
| Public IP | Required (for Jump Server) | No Public IP required for VMs |
| Subnet | Placed in Public Subnet | Deployed in AzureBastionSubnet |
| Access Method | RDP/SSH via Jump Server | RDP/SSH via Azure Portal (HTTPS) |
| Security | Higher attack surface (public IP exposed) | No public IP on VMs – reduced attack surface |
| Management | You manage OS, patching, updates | Fully managed by Microsoft |
| Cost | Pay for VM, Storage, IP, and more | Pay-as-you-go, no VM maintenance cost |
| Use Case | Legacy setup or custom access needs | Modern, secure, scalable, zero public IP |

## SUBNETS EXPLAINED

**AzureBastionSubnet**
Dedicated subnet for Azure Bastion. Must be named exactly as "AzureBastionSubnet".

**App Subnet**
Holds application tier resources (VMs, App Services, Containers, etc.)

**DB Subnet**
Holds database tier resources. Usually kept isolated for security.

**Web Subnet**
Holds web tier resources (VMs, Load Balancers, Web Apps, etc.)

**Subnets help in network segmentation, security control (NSG), and traffic isolation.**

## KEY TAKEAWAY

- Jump Server requires manual management and has a public IP.
- Azure Bastion is fully managed and keeps your VMs private and secure.
- Azure Bastion requires a dedicated subnet named AzureBastionSubnet.

## INTERVIEW TIP

Always remember: Azure Bastion requires a dedicated subnet named AzureBastionSubnet. Without it, deployment will fail.

# TERRAFORM DRIFT STATE

When real infrastructure is different from your code & state file

## 1. DESIRED STATE (CODE)

What you define in Terraform configuration

```
resource "azurerm_storage_account"
 "storage" {
  name   =    = "mystorage123"
  account_tier   = "Standard"
  replication_type = "LRS"
}
```

main.tf

## 2. STATE FILE

What Terraform thinks is actually created

```
{
 "name": "mystorage123",
 "account_tier": "Standard",
 "replication_type": "LRS"
}
```

terraform.tfstate

## 3. REAL INFRASTRUCTURE

What actually exists in the cloud

Storage Account : mystorage123
Account Tier : Premium
Replication Type : LRS

**DRIFT DETECTED**

Actual infrastructure is different from code/state

## HOW TERRAFORM DETECTS DRIFT?


Run terraform plan

Terraform compares code, state file & real infrastructure

Shows the difference & required changes

## EXAMPLE

- You create a Storage Account using Terraform
- Someone changes the tier from Standard to Premium in Azure Portal
- Terraform code & state file are still showing Standard
- Now, Terraform detects drift

## BEST PRACTICES

- Avoid manual changes in the cloud console
- Manage infrastructure only through Terraform
- Use remote state with state locking
- Run terraform plan before every apply
- Regularly review infrastructure changes

**KEY TAKEAWAY:** Terraform works best when it is the **single source of truth** for your infrastructure. Avoid manual changes to keep your infrastructure consistent, predictable & easy to manage.

Terraform

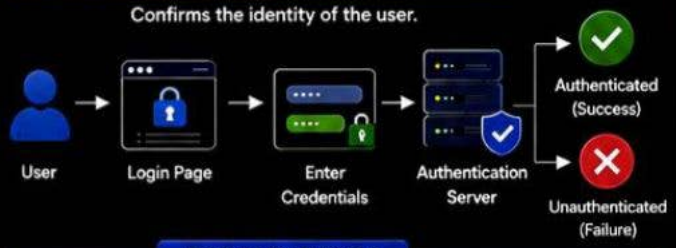
# AUTHENTICATION VS AUTHORIZATION

Authentication verifies WHO you are. Authorization determines WHAT you can do.

Understanding the difference between Authentication and Authorization is essential for building secure Cloud, DevOps, and Cybersecurity environments.

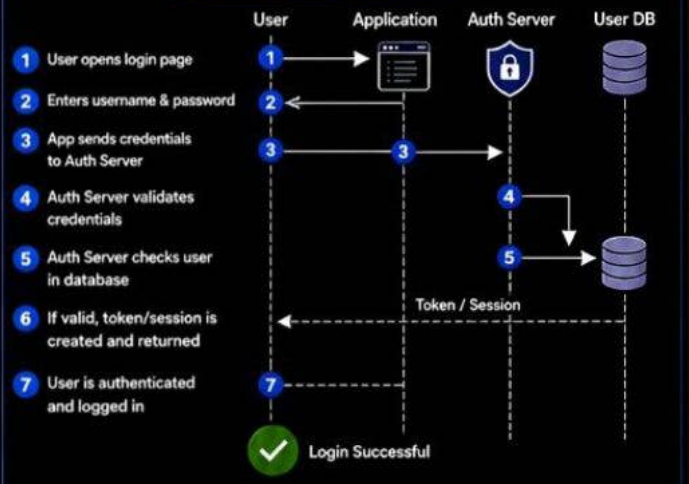
## 1. AUTHENTICATION (Verify WHO You Are)

Confirms the identity of the user.



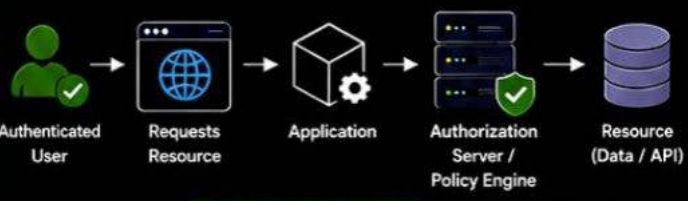

### Authentication Methods


### Authentication Flow (Login)

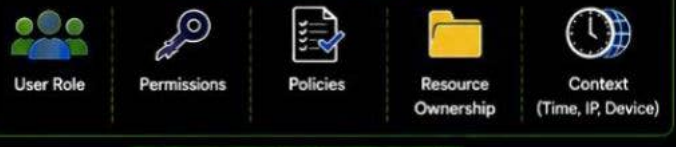



## 2. AUTHORIZATION (Verify WHAT You Can Do)

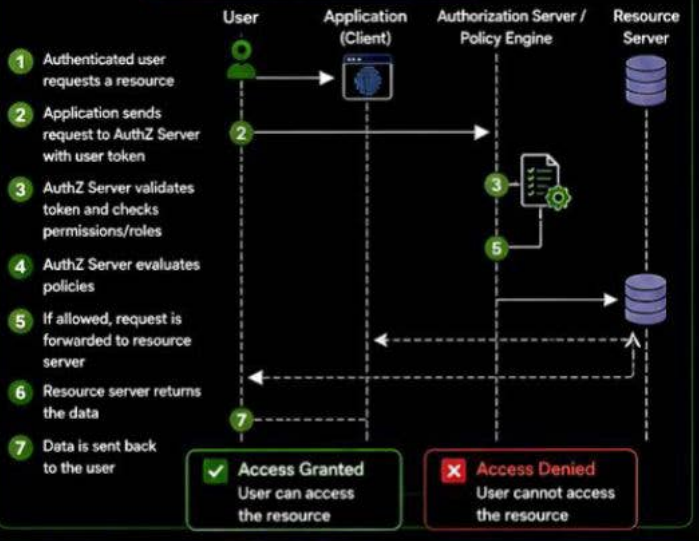
Determines what resources and actions the user is allowed to access.



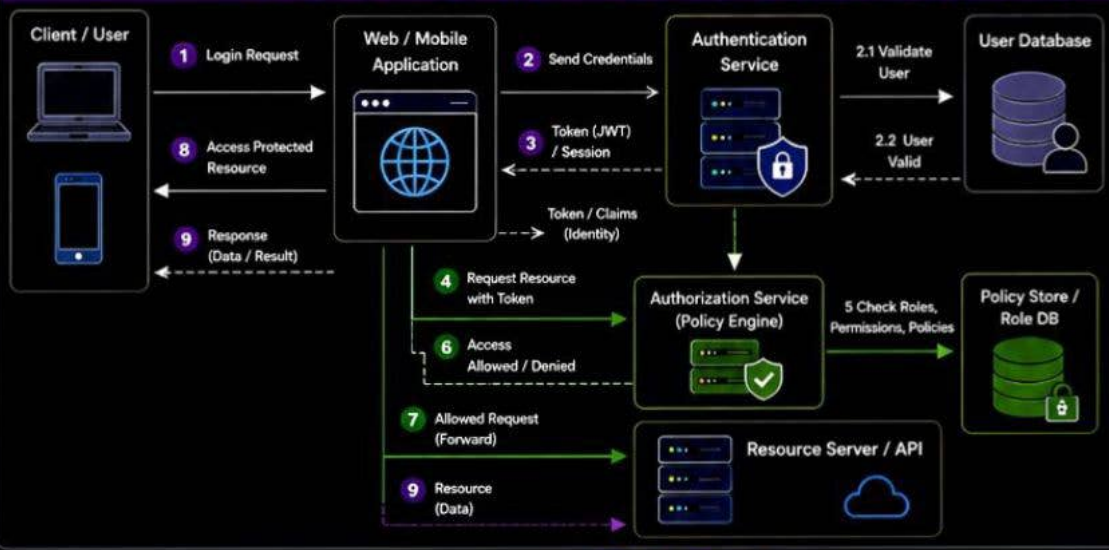

### Authorization Factors




### Authorization Flow (Access Control)




## 3. END-TO-END ARCHITECTURE: AUTHENTICATION vs AUTHORIZATION




**Authentication = WHO are you?**

- Verifies identity
- Issues token/session

**Authorization = What can you do?**

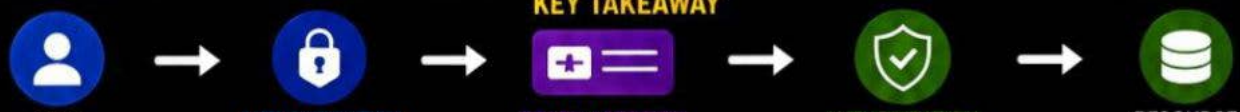
- Verifies permissions
- Grants or denies access

**Token (Example JWT Claims)**

```
{
  "sub": "user123",
  "role": "admin",
  "permissions": ["read:dashboard",
                  "write:user"],
  "exp": 1716389380
}
```

Token is proof of authentication used for authorization.

## KEY TAKEAWAY



**USER**
Person trying to access a system

**AUTHENTICATION (WHO?)**
Verifies identity (Who you are)

**TOKEN / SESSION (PROOF)**
Proof of successful authentication

**AUTHORIZATION (WHAT?)**
Determines access (What you can do)

**RESOURCE (ACCESS)**
Grants access to requested resource)

# RAG vs FINE-TUNING vs PROMPT ENGINEERING

One of the most common questions in AI:

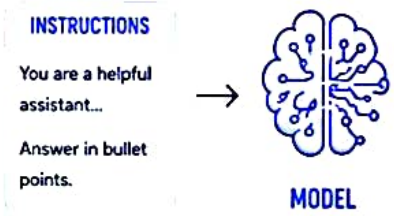
**Should I use prompting, RAG, or fine-tuning?**

**The answer depends on what you're trying to change.**

◆ By-Aadit Gupta ◆

## 1 PROMPT ENGINEERING

Change the instructions, not the model.




**BEST FOR:**

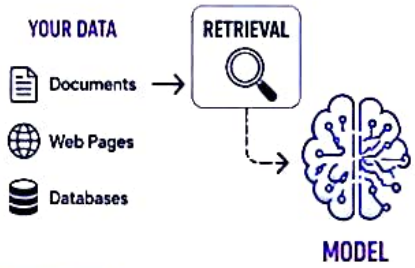
- Controlling behavior
- Improving output quality
- Formatting and reasoning
- Quick experimentation

**USE IT WHEN**

The model already has the knowledge you need.

## 2 RAG

Give the model access to external knowledge at runtime.




**BEST FOR:**

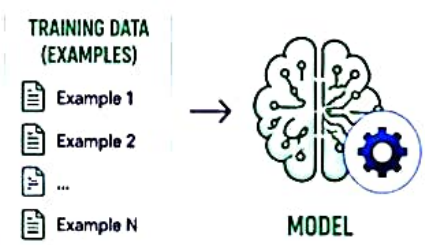
- Private or enterprise data
- Frequently changing information
- Reducing hallucinations
- Grounded answers

**USE IT WHEN**

The problem is mainly about giving the model the right context.

## 3 FINE-TUNING

Train the model on examples to change how it behaves.




**BEST FOR:**

- Specialized behavior
- Consistent style or format
- Domain-specific tasks
- High-volume repetitive workloads

**USE IT WHEN**

Prompting alone isn't enough to achieve the desired behavior.

### THE SIMPLEST WAY TO REMEMBER IT:

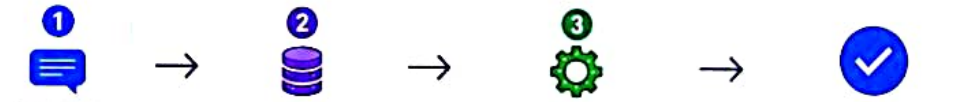
**PROMPTING** → Change the instructions

**RAG** → Change the context

**FINE-TUNING** → Change the behavior

### AND THEY AREN'T MUTUALLY EXCLUSIVE.

A production AI system can use all three:




### THE BIGGEST MISTAKE?

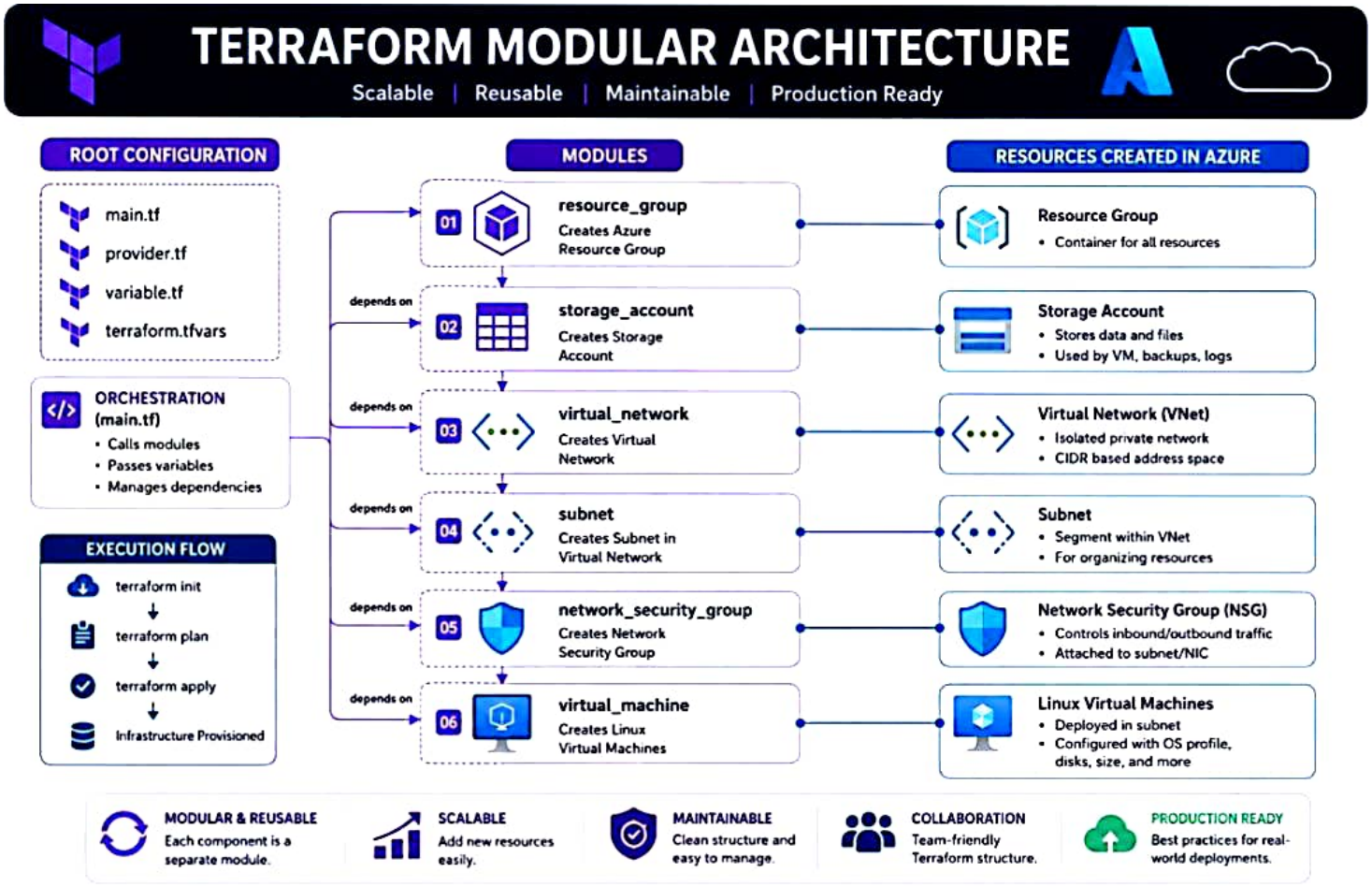
Fine-tuning when the real problem is missing context.

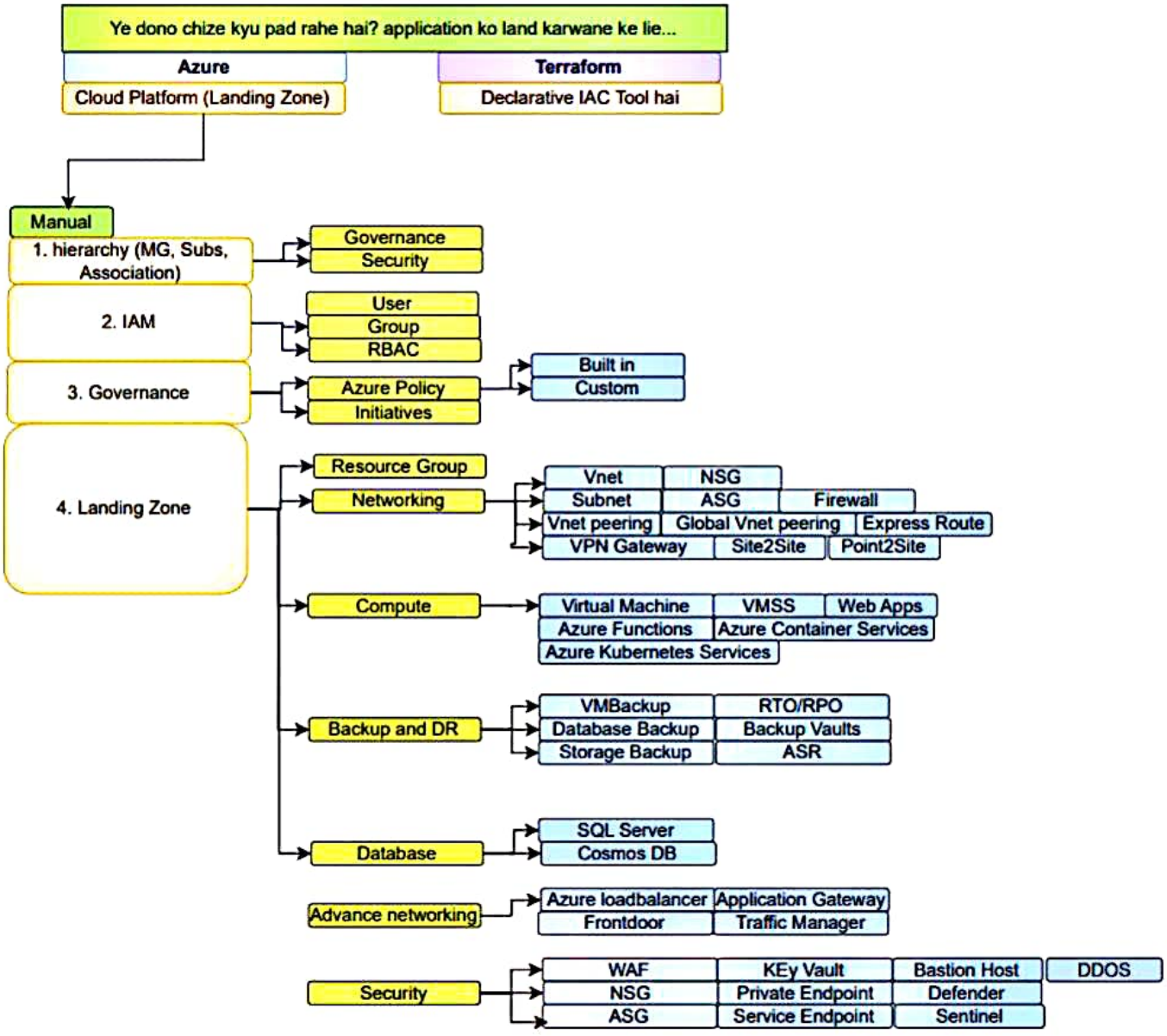
If the model doesn't know something, give it the right information first (RAG / Better context).

If it knows the information but doesn't behave the way you need, then consider fine-tuning.

**Choose the simplest approach that solves the problem.**

**Which approach do you use most in your AI projects?**


TERRAFORM MODULAR ARCHITECTURE
Scalable | Reusable | Maintainable | Production Ready
ROOT CONFIGURATION
main.tf
provider.tf
variable.tf
terraform.tfvars
ORCHESTRATION (main.tf)
Calls modules
Passes variables
Manages dependencies
EXECUTION FLOW
terraform init
terraform plan
terraform apply
Infrastructure Provisioned
MODULES
01
resource_group
Creates Azure Resource Group
depends on
02
storage_account
Creates Storage Account
depends on
03
virtual_network
Creates Virtual Network
depends on
04
subnet
Creates Subnet in Virtual Network
depends on
05
network_security_group
Creates Network Security Group
depends on
06
virtual_machine
Creates Linux Virtual Machines
RESOURCES CREATED IN AZURE
Resource Group
Container for all resources
Storage Account
Stores data and files
Used by VM, backups, logs
Virtual Network (VNet)
Isolated private network
CIDR based address space
Subnet
Segment within VNet
For organizing resources
Network Security Group (NSG)
Controls inbound/outbound traffic
Attached to subnet/NIC
Linux Virtual Machines
Deployed in subnet
Configured with OS profile, disks, size, and more
MODULAR & REUSABLE
Each component is a separate module.
SCALABLE
Add new resources easily.
MAINTAINABLE
Clean structure and easy to manage.
COLLABORATION
Team-friendly Terraform structure.
PRODUCTION READY
Best practices for real-world deployments.


Ye dono chize kyu pad rahe hai? application ko land karwane ke lie...
Azure
Cloud Platform (Landing Zone)
Terraform
Declarative IAC Tool hai
Manual
1. hierarchy (MG, Subs, Association)
Governance
Security
2. IAM
User
Group
RBAC
3. Governance
Azure Policy
Initiatives
Built in
Custom
4. Landing Zone
Resource Group
Networking
Vnet
NSG
Subnet
ASG
Firewall
Vnet peering
Global Vnet peering
Express Route
VPN Gateway
Site2Site
Point2Site
Compute
Virtual Machine
VMSS
Web Apps
Azure Functions
Azure Container Services
Azure Kubernetes Services
Backup and DR
VMBackup
RTO/RPO
Database Backup
Backup Vaults
Storage Backup
ASR
Database
SQL Server
Cosmos DB
Advance networking
Azure loadbalancer
Application Gateway
Frontdoor
Traffic Manager
Security
WAF
KEy Vault
Bastion Host
DDOS
NSG
Private Endpoint
Defender
ASG
Service Endpoint
Sentinel